Name: Jackson Cahn, Jacob Martinez, Laith Titouah, Marcus Razo, Eldrick Hsiu
Date: 10/27/2026
Project: MD-Gooey

Problem 1

Write a short paragraph to answer these three questions:

- What are the two major concerns of any software project?
- Which of those two do you feel is more important?
- Where does the idea of complete functionality fit with these two concerns?

The two major concerns of any software project are how much it will cost and how long it will take. I think time is generally more important because delays can affect the customer's ability to use the software when they actually need it, although cost is still an important limitation. Complete functionality relates to both concerns because implementing every requested feature requires additional time and resources. As the book explains, great software development is about delivering what the customer needs, on time, and within budget; this means prioritizing essential functionality rather than assuming every possible feature must be completed in the initial release.

Problem 2

Write a short paragraph to answer these three questions and briefly explain your opinion:

- In the Agile method for software development, what are the five main phases that occur in each and every iteration?
- Do you feel that any of them could be done at the start of the project and not be repeated in every iteration?
- Do you feel that would save time overall on the project?

The five main phases of each Agile iteration are requirements, design, development, testing, and delivery. I think some requirements gathering and high-level design can be done at the beginning of the project, especially when the overall goals are already understood. However, these phases should still be revisited during each iteration because customer requirements can change, and new problems may emerge during development. Although skipping repeated planning or design might save time initially, I think it would ultimately create more work if the team discovers that its original assumptions were incorrect.

Problem 3

Write a short paragraph to answer these four questions and briefly explain your opinion:

- In the Waterfall method for software development, what are the main phases that occur?
- How are they different from the phases in the Agile method?
- What other phases are in Waterfall that are left out of Agile?
- Do you think these are needed in Waterfall?
- Describe a situation using Agile in which one of these extra Waterfall phases might be needed.

The Waterfall method follows a sequential process of requirements gathering, design, implementation, testing, deployment, and maintenance. Unlike Agile, which repeats its development phases in smaller

iterations, Waterfall generally completes each phase before moving to the next. Waterfall also places greater emphasis on upfront planning, formal documentation, and distinct deployment and maintenance stages, although these activities are not necessarily absent from Agile. I think these additional stages are important in Waterfall because its structure makes changes more difficult later in development. However, Agile projects may still need similar processes; for example, a web application using university authentication might require formal security documentation and approval before deployment.

Problem 4

Write one-sentence answers to the following questions:

- What is a user story?

A story about how the software will be used from the customer's perspective.

- What is blueskying?
- What are four things that user stories SHOULD do?

User stories should describe one thing the software will do for the customer, be written in a language the customer understands, be brief and written by the customer.

- What are three things that user stories SHOULD NOT do?

They should not be lengthy, using terms the customer doesn't understand, or mention specific technologies.

- Does the Waterfall method have user stories?

Waterfall typically relies on more formal requirements and use cases, but user stories can still be used to help gather and communicate customer requirements.

Problem 5

What is your opinion on the following statements, and why do you feel that way:

> All assumptions are bad,
> and no assumption is a
> good assumption.

I don't agree that all assumptions are bad. In the early stages of development, or at any point where the details of the project are unknown, it is often necessary to make assumptions to make progress. The issue with assumptions are when they remain unverified and are treated as fact. As customer needs and requirements change, developers should identify and challenge their assumptions, especially when estimates differ significantly between team members. This allows developers to move forward through changing conditions while still making sure that their understanding of the requirements aligns with the customer's expectations.

A big user story estimate is
a bad user story estimate.

I agree because large user stories can be too broad and reflect a misunderstanding of requirements. The book emphasizes breaking larger stories into smaller, more manageable pieces, which makes them easier to estimate and implement within an iteration. However, I don't think a large estimate is necessarily inaccurate; rather, it should be treated as an indication that the story needs further examination before development begins.

Problem 6

Fill in the blanks in the statements below, using the following things [you can use each thing for more than one statement]: Blueskying; Role playing; Observation; User story; Estimate; Planning poker.

- You can dress me up as a use case for a formal occasion: User Story
- The more of me there are, the clearer things become: User Story
- I help you capture EVERYTHING: Blueskying, Observation
- I help you get more from the customer: Roleplaying, Observation
- In court, I'd be admissible as firsthand evidence: Observation
- Some people say I'm arrogant, but really I'm just about confidence: Estimate
- Everyone's involved when it comes to me: Blueskying (not planning poker)

NOTE: when you have finished, check your answers with the result in your text on page 62. Do you agree with the book answers? If you disagree with any of them, justify your preferred answer.

I agree with most of the book's answers because each technique contributes to understanding the customer's requirements. I especially agree that observation helps capture information that might otherwise be overlooked, since customers may not always describe everything they do. However, I initially thought planning poker would fit the final statement because it involves the entire development team. The book instead identifies blueskying because it includes both developers and customers, while planning poker is primarily a developer activity. That distinction makes sense.

Problem 7

Explain what is meant by a better than best-case estimate.

A better-than-best-case estimate is an estimate that assumes a task can be completed in even less time than the most optimistic realistic scenario. This often happens when developers estimate only the time spent programming without accounting for debugging, testing, meetings, and unexpected problems. The book distinguishes between ideal development time and actual development time through the concept of velocity.

Problem 8

In your opinion, when would be the best time to tell your customer that you will NOT be able to meet her delivery schedule? Why do you feel that is the best time? Do you think that would be a difficult conversation? If so, how could you make it less difficult?

I think the best time to tell a customer that their delivery schedule cannot be met is as soon as the team has enough information to recognize that the deadline is unrealistic. Waiting until the deadline approaches only makes the situation worse because the customer has less time to adjust their expectations. I think this would be a difficult conversation, especially if the customer has already made plans around the expected delivery date. However, it would be easier if the developers could provide clear estimates and offer alternatives, such as reducing the scope of the next release or adjusting the deadline. As the book emphasizes, working with the customer to prioritize functionality is important because it allows the team to deliver the most valuable features within the available time.

Problem 9

Write a short paragraph to discuss why you think branching in your software configuration is bad or good, then describe a scenario to support your opinion.

I think branching is generally a good practice in software development because it allows developers to maintain different versions of their software without interfering with the main codebase. For example, when for our current product we are maintaining a separate branch that can be used to implement authentication while the main branch continues to host a stable version of the front-end website. However, as the book discusses, branching can also introduce problems because maintaining multiple codebases increases the work needed to keep them synchronized. If branches remain separate for too long, merging changes can become increasingly difficult. I think branching is most useful when there is a clear reason to maintain separate versions, and developers should avoid creating unnecessary branches.

Problem 10

Have you used a build tool in your development? If you have, which tool have you used? What are its good points and bad points — in other words, what do you like about it and/or dislike about it?

I have used Vite as a build tool for developing a React web application. One thing I like about Vite is that it simplifies the development and build process by automatically handling the bundling and optimization of JavaScript and TypeScript code. It also provides a fast development server that makes testing frontend changes convenient. However, configuring builds and troubleshooting dependency issues can sometimes be frustrating, especially when deploying the application to different environments. Overall, I think Vite is useful because it automates much of the build process and allows developers to focus more on implementing functionality.